作业

305

(1). 事务是用户定义的一个数据库操作序列,这些操作要么全做要么全不做,事务是一个不可分 割的工作单元。事务具有 4 个特性:原子性、一致性、隔离性和持续性。

原子性：事务是数据库的逻辑工作单位，事务中包括的诸操作要么都做，要么都不做。

一致性：事务执行的结果必须是使数据库从一个一致性状态变到另一个一致性状态

隔离性：一个事务的执行不能被其他事务干扰。即一个事务内部的操作及使用的数据 对其他并发事务是隔离的，并发执行的各个事务之间不能互相干扰。

持续性：持续性也称永久 性(17ermanence)，指一个事务一旦提交，它对数据库中数据的改变 就应该是永久性的。接下来的其他操作或故障不应该对其执行结果有任何影响。

(2).

事务执行的结果必须是使数据库从一个一致性状态变到另一个一致性状态。如果数据库系 统运行中发生故障，有些事务尚未完成就被迫中断，这些未完成事务对数据库所做的修改有一 部分已写入物理数据库，这时数据库就处于一种不正确的状态，或者说是不一致的状态。

举列

假设你在一个在线购物网站上购买商品。你的订单涉及两个事务操作：首先，从你的银行账户中扣除购买金额，然后更新商品库存以反映你的购买。但是，如果在扣款后、库存更新前，系统遭遇到故障，事务将被中断。

此时，购买金额已经从你的账户中扣除，但商品库存并没有相应地减少。结果就是，虽然你的钱已经被扣走了，但商品却没有被准确地更新，导致库存显示和实际库存不一致。这可能会导致购物网站显示商品为售罄，但实际上仍有库存，或者客户下单后发现商品实际已售罄，从而破坏了用户体验。

因此，确保事务执行是从一个一致性状态到另一个一致性状态的关键。在这种情况下，一致性状态是指购物网站的库存数量与实际库存数量相匹配，而故障中断导致了不一致的状态。

326p

(1).

并发控制在数据库中是为了处理多用户同时访问和修改数据时可能出现的问题。这些技术确保了事务具有原子性、一致性、隔离性和持久性这些特性。简而言之，它们保证了数据库操作的正确性和可靠性。

(2).

并发操作可能导致数据不一致，包括丢失修改、脏读、不可重复读和幻读。为了避免这些问题，可以使用锁机制、事务隔离级别、多版本并发控制和乐观并发控制等方法。这些技术能够确保在多用户同时操作数据库时，数据保持一致性和可靠性。

(3).
封锁是一种并发控制机制，用于管理多个事务对数据库中共享数据的访问。基本的封锁类型有共享锁和排他锁。共享锁允许多个事务同时读取数据，而排他锁只允许一个事务对数据进行读取和修改。共享意向锁和排他意向锁用于指示事务计划获取共享锁或排他锁的意图。这些封锁类型确保事务之间相互协调，防止并发操作导致的数据不一致性。

(4).

封锁机制通过对数据进行加锁，防止并发事务同时修改同一数据，从而确保数据的一致性。通过排他锁防止丢失修改，通过共享锁防止脏读和不可重复读，通过共享锁或锁定整个范围防止幻读。这样，封锁机制保证了事务之间的数据访问有序、一致和可控。

(6).

死锁是指多个事务相互等待对方持有的资源而无法继续执行的情况。预防死锁的方法包括设定加锁顺序、超时机制、死锁检测与回滚、使用锁超时和避免过度使用锁等。这些方法可以有效地减少死锁的发生，提高系统的稳定性和性能。

(7).

一种检测死锁的方法是使用等待图，如果等待图中存在环路，则发生死锁。解除死锁的方法包括回滚事务、资源剥夺和事务回滚与重启。这些方法可以有效地解除死锁并恢复系统的正常运行。

(8).

正确的并发调度需要满足隔离性、一致性、原子性和持久性。这意味着每个事务都应该在独立的执行环境中操作数据，数据库在事务执行过程中应该保持数据的完整性和约束条件，事务要么全部执行成功，要么全部失败回滚，并且已提交的事务对数据库的修改应该是持久的。实现正确的并发调度需要结合合适的并发控制机制，同时考虑系统的性能和资源利用率。

9.

(1)

结果1：T1->T2->T3：A = ((0+2)*2)^2 = 16

结果2：T1->T3->T2：A = (0+2)^2*2 = 8

结果3：T2->T1->T3：A = (0*2+2)^2 = 4

结果4：T2->T3->T1：A = (0*2)^2+2 = 2

结果5：T3->T1->T2：A = (0+2)^2*2 = 8

结果6：T3->T2->T1：A = (0*2)^2+2 = 2

(2)

一个可串行化的调度可以是按照T1、T2、T3的顺序依次执行。这样执行的结果是：

A = 0+2 = 2 (T1)

A = 2*2 = 4 (T2)

A = 4^2 = 16 (T3)

(3)

一个非串行化的调度可以是T2、T1、T3的顺序。这样执行的结果是：

A = 0*2 = 0 (T2)

A = 0+2 = 2 (T1)

A = 2^2 = 4 (T3)

(4)

由于T1、T2、T3都是对A进行读写操作，可以采用下面的顺序：

T1：获取A的排他锁，执行A=A+2，释放A的锁

T2：获取A的排他锁，执行A=A*2，释放A的锁

T3：获取A的排他锁，执行A=A^2，释放A的锁

(5)

一个可能产生死锁的调度是：

T1：获取A的共享锁

T2：获取A的共享锁

T3：获取A的排他锁（等待）

(10).

该调度是冲突可串行化的调度。因为该调度中所有的操作都是按照事务的顺序进行的，不 存在交错操作，所以该调度是冲突可串行化的。

(12).

假设有三个事务 T1, T2, T3，它们对数据库中的两个数据项 A 和 B 进行操作。其执行序列 为：

T1: R(A); W(B); T2: R(B); W(A); T3: R(A); W(B);

这个并发调度不遵守两段锁协议，因为 T2 在获取 B 的锁之前已经获取了 A 的锁。然而，这个 并发调度是可串行化的。我们可以将其分解成以下两个子调度：

子调度 1：T1; T3; 子调度 2：T2;

子调度 1 和子调度 2 都是可串行化的，因此原来的并发调度也是可串行化的。

13.

(1) 正确的调度是指不违反事务的 ACID 属性，同时满足数据库的一致性约束条件的调度。 (2) 可串行化的调度是指与某个串行调度等价的调度。

(3) 遵守两阶段封锁（2PL）的调度是指所有事务

都遵守两阶段封锁协议的调度.

(4) 串行调度是指所有事务串行执行的调度。

14.

(1)

改写后的 T1 和 T2 如下：

T1: lock(A); R(A); lock(B); R(B); B = A + B; W(B); unlock(B); unlock(A)

T2: lock(B); R(B); lock(A); R(A); A = A + B; W(A); unlock(A); unlock(B)

(2)

T1 和 T2 的执行可能会引起死锁。例如，如果 T1 获得了 A 的锁，T2 获得了 B 的锁，那么 它们都无法继续执行下去，就会发生死锁。一个可能的调度是：

T1: lock(A); R(A); lock(B); R(B); B = A + B; W(B); unlock(B); unlock(A)

T2: lock(B); R(B); lock(A); R(A); A = A + B; W(A); unlock(A); unlock(B)

这是一个正确的调度，因为它没有违反事务的 ACID 属性，也满足数据库的一致性约束条件。 同时，它也是可串行化的调度，因为它等价于以下的串行调度：

T1: lock(A); R(A); lock(B); R(B); B = A + B; W(B); unlock(B); unlock(A)

T2: lock(A); R(A); lock(B); R(B); A = A + B; W(A); unlock(B); unlock(A)